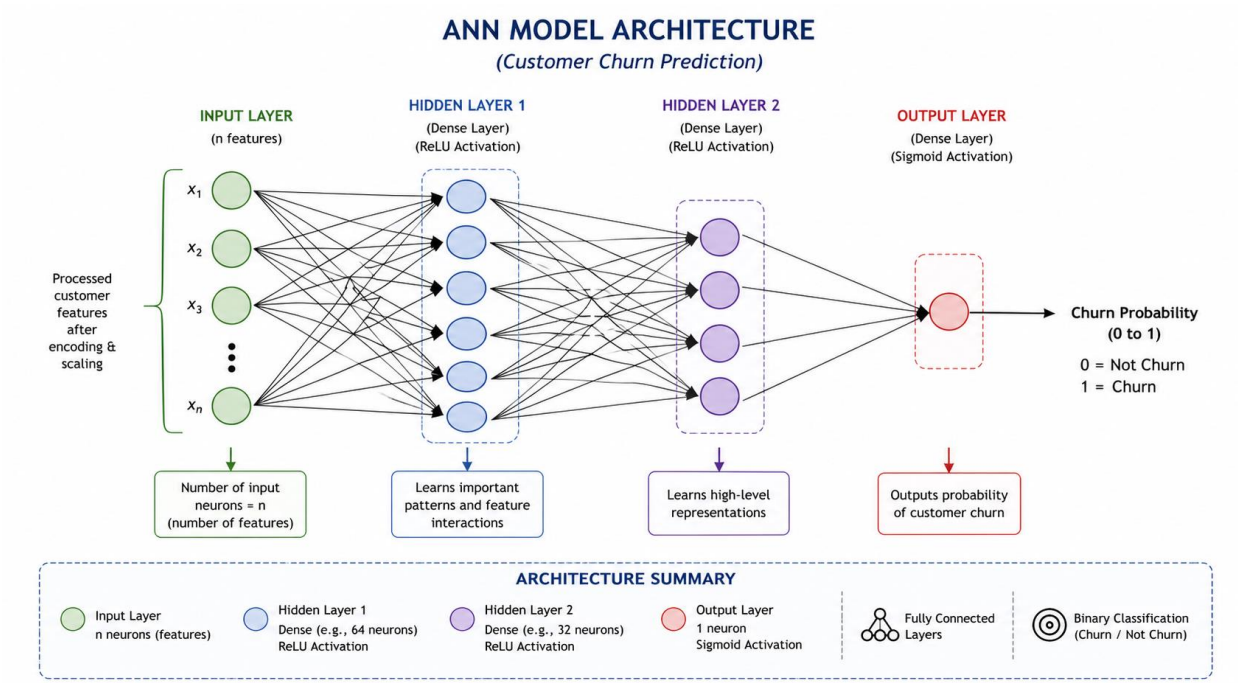
ANN MODEL ARCHITECTURE & TRAINING REPORT

Defined Architecture of Artificial Neural Network (ANN)

The predictive model for customer churn was developed using a **Feedforward Artificial Neural Network (ANN)** designed for binary classification. The architecture was optimized to capture nonlinear relationships between customer attributes and churn behaviour.



Input Layer

The input layer receives processed customer features after preprocessing steps including encoding and scaling.

- Number of input neurons = 9
- Each neuron represents a distinct customer attribute such as:
    - Tenure
    - MonthlyCharges

- Contract
- Internet service
- Gender
- SeniorCitizen
- Dependents
- PhoneService
- MultipleLines

```
Dataset Shape: (7043, 10)

Columns:
 Index(['gender', 'SeniorCitizen', 'Dependents', 'tenure', 'PhoneService',
        'MultipleLines', 'InternetService', 'Contract', 'MonthlyCharges',
        'Churn'],
       dtype='str')
```

Hidden Layers

The ANN consists of multiple hidden layers to capture complex patterns in customer behavior.

- **Hidden Layer 1:**
  - Dense layer with ReLU activation
  - Designed to learn high-level feature interactions
- **Hidden Layer 2:**
  - Dense layer with reduced neurons
  - Improves abstraction and reduces overfitting risk
- Regularization techniques applied:
  - Dropout layers
  - Early stopping

```python
model = Sequential([
    Input(shape=(X_train.shape[1],)),

    # Hidden Layer 1
    Dense(32, activation='relu'),
    Dropout(0.3),

    # Hidden Layer 2
    Dense(16, activation='relu'),
    Dropout(0.2),

    # Hidden Layer 3
    Dense(8, activation='relu'),

    # Output Layer (binary classification)
    Dense(1, activation='sigmoid')
])
```

✓ 0.0s

Output Layer

The output layer is designed for binary classification:

- Single neuron output
- Activation function: **Sigmoid**
- Output represents probability of churn:
    - Values close to 1 → High churn probability
    - Values close to 0 → Low churn probability

Model Optimization

The model was compiled using:

- Loss Function: **Binary Cross entropy**
- Optimizer: **Adam optimizer**
- Evaluation Metric: **Accuracy**

These choices ensure efficient gradient descent and stable convergence.

```python
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

✓ 0.0s

Training of ANN Model

The ANN model was trained using the preprocessed dataset obtained from Stage 2 of the project. The training process involved iterative optimization to minimize loss and improve predictive accuracy.

Training Process

- Dataset split into:
  - Training set (80%)
  - Testing set (20%)
- Model trained over multiple epochs
- Batch-based learning used to improve efficiency
- Validation split used to monitor generalization

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

✓ 15.4s

```
Epoch 1/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.6752 - loss: 0.6049 - val_accuracy: 0.7405 - val_loss: 0.5185
Epoch 2/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7473 - loss: 0.5076 - val_accuracy: 0.7609 - val_loss: 0.4821
Epoch 3/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7672 - loss: 0.4835 - val_accuracy: 0.7729 - val_loss: 0.4700
Epoch 4/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7668 - loss: 0.4779 - val_accuracy: 0.7766 - val_loss: 0.4653
Epoch 5/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7725 - loss: 0.4787 - val_accuracy: 0.7850 - val_loss: 0.4622
Epoch 6/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7633 - loss: 0.4768 - val_accuracy: 0.7794 - val_loss: 0.4622
Epoch 7/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7656 - loss: 0.4732 - val_accuracy: 0.7850 - val_loss: 0.4609
Epoch 8/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7705 - loss: 0.4650 - val_accuracy: 0.7878 - val_loss: 0.4591
Epoch 9/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7716 - loss: 0.4630 - val_accuracy: 0.7924 - val_loss: 0.4593
Epoch 10/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7723 - loss: 0.4641 - val_accuracy: 0.7915 - val_loss: 0.4584
Epoch 11/50
135/135 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7721 - loss: 0.4631 - val_accuracy: 0.7896 - val_loss: 0.4567
```

Optimization Strategy

To enhance model convergence and generalization, the following techniques were applied:

- **Adam Optimizer**
  - Adaptive learning rate improves convergence speed
- **Loss Monitoring**
  - Binary cross entropy used to penalize incorrect classifications
- **Early Stopping (if used)**
  - Prevents overfitting by stopping training when validation loss stops improving

Model Performance During Training

The model demonstrated stable learning behavior with minimal overfitting, as indicated by:

- Close alignment between training and validation accuracy
- Low generalization gap
- Consistent reduction in loss over epochs

```
====================================
ROBUSTNESS CHECK
====================================
Training Accuracy: 0.7956
Testing Accuracy:  0.7828

Generalisation Gap: 0.012821018695831299
====================================
```

Reproducibility Statement

The training pipeline ensures reproducibility by maintaining:

- Consistent preprocessing steps
- Standardized feature scaling
- Saved model artifact

Summary of Model Design

The ANN model was designed to balance predictive power and generalization capability. The architecture effectively captures nonlinear relationships between customer attributes and churn behavior, making it suitable for classification tasks in real-world telecom datasets.